Today

1. Why $3 + 4 \times 2$ is **11**, not 14: **precedence**.
2. Three ways to write an expression: **infix**, **prefix**, **postfix**. (Last class: the three traversals of an expression tree.)
3. **Evaluating** postfix with one stack.
4. **Converting** infix to postfix: the **shunting-yard** algorithm (a stack of operators that wait).

- **Exercise 8 (Shunting-Yard Trace)** is posted today and is due **Thursday, Nov 12**. Today's trace tables are the model for it.
- Project 4 (Expression Evaluator) builds on today's ideas. It is posted later; this class is the preview.

Question: What Does a Calculator Print?

Evaluate $3 + 4 \times 2$ the way standard arithmetic does (the way a calculator that respects precedence would).

(a) Which answer is correct, 11 or 14?
(b) Reading left to right, what is the *first* operator we meet? Why can we not do it yet?

(a) 11. The rule of precedence: $\times$ and $\div$ bind tighter than $+$ and $-$.

(b) We meet $+$ first, but its right operand is not 4 alone: it is 4×2. The $+$ **has to wait** until we have seen what follows.

Analogy: you read the expression aloud to a friend who does the arithmetic. After "three plus..." your friend cannot write anything yet, because what is added to 3 is not known. So they put a **sticky note** "3+" aside, finish $4 \times 2 = 8$, and only then pick the note back up: $3 + 8 = 11$. If they added at once they would get $(3 + 4) \times 2 = 14$. **Waiting** is the whole idea of today.

Three Notations

The operator can go **between**, **before**, or **after** its operands.

Infix (operator between)	**Prefix** (before)	**Postfix** (after)
`3 + 4`	`+ 3 4`	`3 4 +`
`3 + 4 * 2`	`+ 3 * 4 2`	`3 4 2 * +`
`(3 + 4) * 2`	`* + 3 4 2`	`3 4 + 2 *`

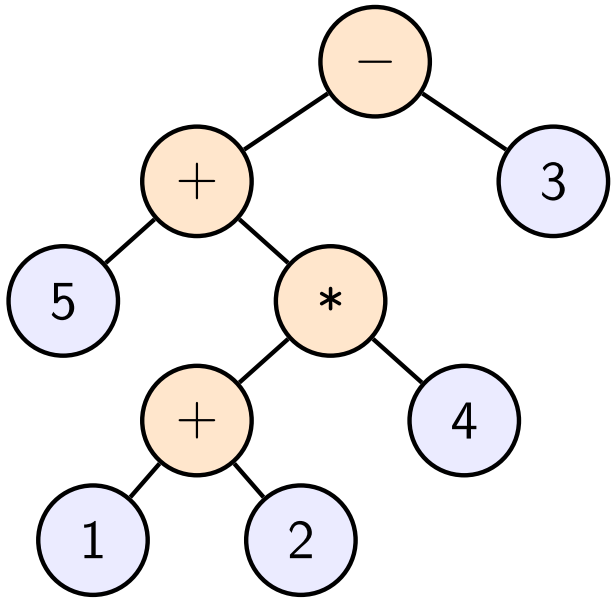
Question: write `(3 + 4) * 2` in postfix first, then compare. Notice that **postfix has no parentheses and no precedence rules**: the order of the tokens already says what to do first. That is why a stack can evaluate it with no precedence decisions to make.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree for `5 + (1 + 2) * 4 - 3`.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)

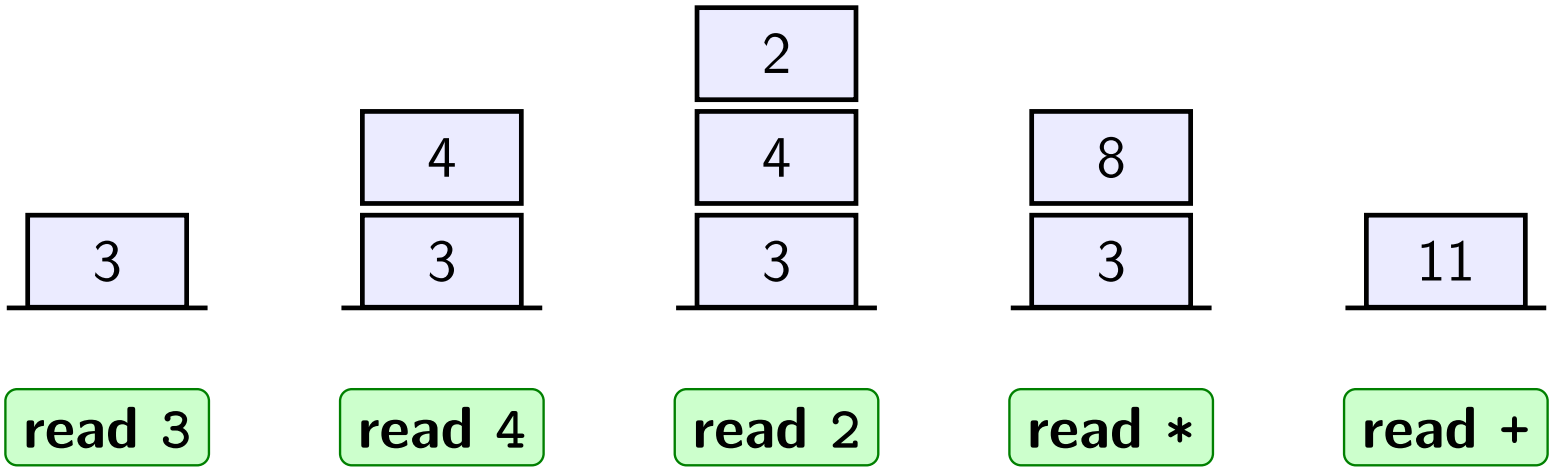


Post-order:	`5 1 2 + 4 * + 3 -`
Pre-order:	`- + 5 * + 1 2 4 3`
In-order, no parentheses:	`5 + 1 + 2 * 4 - 3`
	value 11, **not** the tree's value
In-order, with parentheses:	`((5 + ((1 + 2) * 4)) - 3)`
	value 14, matching the tree

Why parentheses? Plain in-order loses the tree's grouping; parentheses restore it.
Check: the post-order string is the postfix that shunting-yard produces in Trace 3 later in this lecture.

Evaluating Postfix: One Stack

Rule. Read tokens left to right. **Number:** push it. **Operator:** pop two values, apply the operator, push the result.



3 4 2 * +: read *: pop 2, pop 4, push $4 \times 2 = 8$. Read +: pop 8, pop 3, push 11. The answer is the single value left.

Question: Which Operand Comes Off First?

Evaluate the postfix expression `8 2 / 3 -`.

(a) When we read `/`, the stack holds 8 and 2 (the 2 on top). What are a and b in "a / b"?
(b) What is the final answer?

(a) The **first pop is the right operand** $b = 2$, the second pop is the left operand $a = 8$. So $8/2 = 4$, not $2/8$.

(b) Push 4. Push 3. Read `-`: pop $b = 3$, pop $a = 4$: $4 - 3 = \mathbf{1}$.

For $+$ and $\times$ the order does not matter, so a wrong order **passes the easy tests**. It fails on $-$ and $/$.

Tokenizer 1: Input and Output

A **tokenizer** cuts an expression string into pieces (**tokens**) that the next steps can use. Input is one `std::string`.

input

1	2	+	3	*	(	4	-	1	)

output

"12"	"+"	"3"	"*"	"("	"4"	"-"	"1"	")"

Answer. Nine tokens. Note that 12 is **one** token, not 1 and 2.

Q. What should `tokenize` return?

(A) `std::vector<char>`

(B) `std::vector<int>`

(C) **`std::vector<std::string>`**

(D) `std::string`

Answer: (C). A token is one or more characters ("12", "+"), so a `char` is too small and a `std::string` per token fits; a vector keeps them in order. (B) cannot store +. (D) is the input again, still one big string.

Tokenizer 2: Three Kinds of Characters

Q. Look at `12+3*(4-1)` with spaces allowed, e.g. `"12 + 3"`. How many **categories** of characters must the loop handle?

Category	Characters	What the loop does
1. Space	`' '`	skip it: `++i`, no token
2. Digit	`0...9`	gather all digits: one token
3. Operator or parenthesis	`+ - * / ( )`	one char = one token

Q. For category 3, `text[i]` is the `char` `'+'`. Which expression gives a `std::string` equal to `"+"`?

(A) `text[i]`

(B) `std::string(1, text[i])`

(C) `std::to_string(text[i])`

(D) `std::string(text[i])`

Answer: (B). `std::string(1, c)` builds a string of **one copy** of `c`. (A) is a `char`, not a string. (C) turns the character's *code* into digits: in ASCII, `to_string('+')` is `"43"`. (D) has no constructor taking a single `char`.

Tokenizer 3: Why `while`, Not `for`?

Trace `tokenize("12+3")`. Each step starts at index `i` and reads **one token**.

Step	`i` before	What starts at `i`	token	`i` after
1	0	digits 1, 2	`"12"`	**2** (moved by 2)
2	2	+	`"+"`	3 (moved by 1)
3	3	digit 3	`"3"`	4 (moved by 1)

Q. A typical `for (i = 0; i < size; ++i)` loop adds **1** to `i` every pass. Which statement best explains our choice of a `while` loop with our own `i`?

(A) A `for` loop cannot index a `std::string`.

(B) **Each branch controls how far `i` advances: 1 for a space or operator, several for a multi-digit number.**

(C) A `while` loop runs faster than a `for` loop.

(D) A `for` loop cannot call `push_back`.

Answer: (B). After reading `"12"` we must continue at the +, not at the 2 we already used. We use `while` so that each branch controls how far `i` advances.

Tokenizer 4: When Do We Stop Collecting Digits?

The index `i` stands on the first digit of a number. We take digits one at a time. Try two inputs:

Input	`i`	`text[i]`	**Action**
`"12+3"`	0, 1	`'1'`, `'2'`	digits: take them
	2	`'+'`	**not a digit**: stop (number is `"12"`)
`"12"`	2	`i == size`	**string ended**: stop (no more input)

Q. Finish the sentence: "Keep collecting digits . . . "

(A) as long as the current character is not a space.

(B) **as long as there are characters left and the current character is a digit.**

(C) as long as the current character is a digit (nothing else to check).

(D) as long as the current character is an operator.

Answer: (B). Two reasons to stop: the next character is *not* a digit (like +), or the string has *ended*. (C) forgets the second reason: `i == text.size()` means there are no more input characters, and we should say so explicitly. (A) would swallow + and -. In code this becomes `i < text.size() && std::isdigit(text[i])`.

Tokenizer 5: Multi-Digit Code

When `text[i]` is a digit, collect digits into a string `num`, then push `num` **once**. (Shortened: the full code casts to `unsigned char`.)

```
else if (std::isdigit(text[i])) {
    std::string num;
    while ( ??? A ??? ) {
        num += text[i++];      // take this digit, move on
    }
    tokens.push_back(num);     // inside or after the loop?
}
```

Q1. What goes in blank A?

(A) `i < text.size()`
(B) `std::isdigit(text[i])`
(C) `i < text.size() && std::isdigit(text[i])`
(D) `text[i] != ' '`

Answer: (C). Continue *while the next character is a digit*, but test `i < text.size()` **first**: after the last digit of `"12"`, `i == text.size()` means there are no more input characters to tokenize.

Q2. Where must `tokens.push_back(num)` go: (A) inside the loop, after each digit; or (B) after the loop ends?

Answer: (B). Inside the loop we would push `"1"` and then `"12"`. After the loop `num` holds the whole number, so we push it once.

Tokenizer 6: Your Turn: Write `tokenize`

Write `std::vector<std::string> tokenize(const std::string& text)` **on paper**.

Plan your code in this order:

1. create `tokens` and an index `i = 0`;
2. loop: `while (i < text.size()) { ... }`, with these cases inside:
3. space: skip it,
4. digit: collect all digits into `num`, then `push_back(num)`;
5. otherwise: push `std::string(1, text[i])` and move on;
6. return `tokens`.

Test your code by hand:

`"7 + 25"`
→ `{"7", "+", "25"}`

`"(1+2)*10"`
→ `{"(", "1", "+", "2", ")", "*", "10"}`

Check yourself. Every branch of the loop must make `i` **larger**, or the loop never ends. The next slide shows the code.

Code: Tokenize (Check Your Code)

Tokens are **strings**, so that 12 is one token, not two.

*Assumes valid input: nonnegative integers, binary + - * /, balanced parentheses (no unary minus or decimals). The last branch does not check characters.*

```
std::vector<std::string> tokenize(const std::string& text) {
    std::vector<std::string> tokens;
    std::size_t i = 0;
    while (i < text.size()) {
        if (std::isspace(static_cast<unsigned char>(text[i]))) { ++i; }
        else if (std::isdigit(static_cast<unsigned char>(text[i]))) {
            std::string num;
            while (i < text.size() &&
                   std::isdigit(static_cast<unsigned char>(text[i]))) {
                num += text[i++];
            }
            tokens.push_back(num);                    // a whole number, e.g. "12"
        } else {
            tokens.push_back(std::string(1, text[i++])); // assumed to be one of + -
                  * / ( )
        }
    }
    return tokens;
}
// tokenize("12+3*(4-1)") gives {"12", "+", "3", "*", "(", "4", "-", "1", ")"}
```

Evaluate 1: Work an Example

The postfix tokens wait in a **queue**; the values wait in a **stack**. Evaluate `7 2 3 * -` (that is $7 - 2 \times 3$). **Predict** each stack, then reveal.

Step	Token	Queue left	Action	Stack (bottom→top)
1	7	`2 3 * -`	number: push	[7]
2	2	`3 * -`	number: push	[7, 2]
3	3	`* -`	number: push	[7, 2, 3]
4	*	`-`	pop b=3, a=2; push 2×3	[7, 6]
5	-	(empty)	pop b=6, a=7; push $7 - 6$	[1]

Result. The queue is empty and the stack holds **one** value, 1. That value is the answer. Notice step 5: the *first* pop (6) is the *right* operand.

Evaluate 2: The Input and the Return Value

front | "7" | "2" | "3" | "*" | "-" | back

Q1. The tokens are numbers and operators mixed together, such as "7" and "*". What type should the input be?

(A) `std::queue<int>`

(B) `std::queue<char>`

(C) `std::queue<std::string>`

(D) `std::stack<int>`

Answer: (C). A token is text: "12" is more than one character (so not `char`), and "*" is not a number (so not `int`). Both fit in a `std::string`.

Q2. What should `evalPostfix` return?

(A) `std::string`

(B) `bool`

(C) `int`

(D) `double`

Answer: (D). The result is one number. We use `double` so that division is real division: `7 2 /` is 3.5, as in Exercise 8 and Project 4.

Evaluate 3: Why a Queue, Not a Stack?

The postfix tokens were written `7 2 3 * -`. We must process them **in that order**.

Q. Why do we store the input tokens in a **queue**, not a stack?

(A) A stack cannot hold strings.

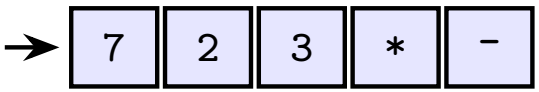
(B) **A queue returns tokens in the order written; a stack returns the *last* token first.**

(C) A queue uses less memory.

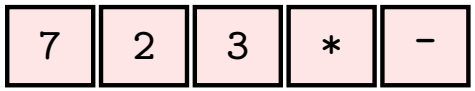
(D) A stack cannot be passed to a function.

Answer: (B). Queue = first in, first out: 7 is read first. The *values* go in a stack, because we need the *most recent* two.

queue: reads 7 first



stack: would read - first



last in, first out: wrong order

Evaluate 4: An Operator, and the End

Q1. The next token read from the queue is an operator, such as `*`. What do we do?

(A) Push the operator onto the value stack.

(B) **Pop two values, apply the operator, and push the result back onto the value stack.**

(C) Pop one value and print it.

(D) Skip it: only numbers matter.

Answer: (B). Pop *b* first (the **right** operand), then *a* (the left), compute *a* op *b*, and push the result. The result can be an operand for the next operator.

Q2. The queue is now empty: every token was processed. What is on the value stack?

(A) Nothing.

(B) **Exactly one value: the answer.**

(C) Every number from the input.

(D) The operators.

Answer: (B). Each operator replaces two values by one, so a valid expression ends with a single value. We return it with `values.top()`. (More than one value, or an empty stack when we need to pop, means the input was not valid postfix.)

Evaluate 5: Your Turn: Write `evalPostfix`

Write `double evalPostfix(std::queue<std::string> postfix)` **on paper**.

Assume syntactically valid postfix input; reject division by zero (`std::runtime_error`).

Plan your code in this order:

1. a `std::stack<double> values`;
2. loop while the queue is not empty: read the front token `t`, then `pop()` the queue;
3. if `t` is an operator: pop `b`, pop `a`, push `a op b`;
4. otherwise: convert `t` to a number (`std::stod`) and push it;
5. after the loop: return `values.top()`.

Test by hand:

`7 2 3 * -` $\rightarrow$ `1`

`8 2 /` $\rightarrow$ `4`

`7 2 /` $\rightarrow$ `3.5`

Check yourself. The **first** pop is the right operand, so `-` gives $a - b$. The next slide shows the code.

Code: Evaluate a Postfix Queue (Check Your Code)

`isOp(t)` is true for `+ - * /`. The queue is passed **by value**: the function consumes its own copy.

```
double evalPostfix(std::queue<std::string> postfix) {
    std::stack<double> values;
    while (!postfix.empty()) {
        std::string t = postfix.front();
        postfix.pop();
        if (isOp(t)) {
            double b = values.top(); values.pop(); // right operand first
            double a = values.top(); values.pop(); // then the left operand
            if (t == "+")     { values.push(a + b); }
            else if (t == "-") { values.push(a - b); }
            else if (t == "*") { values.push(a * b); }
            else { if (b == 0.0) { throw std::runtime_error("division by zero"); }
                   values.push(a / b); }
        } else {
            values.push(std::stod(t));           // a number: push it
        }
    }
    return values.top();                         // the one value left
}
```

Note: `std::stack<double>` means **real division**: `7 / 2` is 3.5, as in Exercise 8 and Project 4.

Dividing by zero is rejected by throwing `std::runtime_error`, as Ex. 8 and Project 4 require.

Assumes valid postfix input.

Converting Infix to Postfix: The Idea

Goal: turn `3 + 4 * 2` into `3 4 2 * +` using a stack, one token at a time.

Question: look at `3 + 4 * 2` and `3 4 2 * +`. (a) Did the **numbers** change order? (b) Which operator was written first, and which came out first?

(a) No: `3, 4, 2` stay in the same order. Numbers can go straight to the output.

(b) $+$ was written first but comes out **last**; $\times$ was written second and comes out first. Operators come out in a **different order**, so they need a place to wait. A **stack** is that place: when operators must leave, the top one leaves first. The precedence and associativity rules decide *when* they leave.

Analogy: a railway **shunting yard**, where cars wait on a side track and are released in the order the schedule needs. That is where the algorithm gets its name.

The Shunting-Yard Rules

Two containers: an **output** list and an **operator stack**. For each token:

Token	**Action**
Number	Append to the output.
Operator o	**While** the top of the stack is an operator with **precedence** $\geq$ that of o: pop it to the output. Then push o.
(	Push it.
)	Pop operators to the output until (is on top; discard the (.
End	Pop everything left on the stack to the output.

Precedence: $* \; / = 2$; $+ \; - = 1$. The operators here are **left-associative**: $8 - 3 - 2$ means $(8 - 3) - 2$. That is why we pop on $\geq$ (equal), not just $>$.

Rules in Action: 8 - 3 - 2

Apply the rules to 8 - 3 - 2, one token at a time. Which rule fires? **Predict** the output and the stack, then reveal.

#	Token	Rule applied	Output	Stack
1	8	number: append to the output	8	
2	-	operator: the stack is empty, nothing to pop; push	8	-
3	3	number: append to the output	8 3	-
4	-	operator: top - has precedence $1 \geq 1$: **pop it to the output**; then push the new -	8 3 -	-
5	2	number: append to the output	8 3 - 2	-
6	*end*	pop everything left	8 3 - 2 -	

Check. 8 3 - 2 -: $8 - 3 = 5$, then $5 - 2 = \mathbf{3}$, which is $(8 - 3) - 2$. Pop on $\geq$ gave the correct grouping.

Trace 1: 3 + 4 * 2

Fill in the last two columns, then check.

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	3	output	3	
2	+	push +	3	+
3	4	output	3 4	+
4	*	push *	3 4	+ *
5	2	output	3 4 2	+ *
6	*end*	pop *	3 4 2 *	+
7	*end*	pop +	3 4 2 * +	

Row 4: $\times$ has **higher** precedence than the $+$ on top, so nothing is popped. The $+$ keeps waiting. At the end, $\times$ is popped before $+$.

Trace 2: Parentheses, `(3 + 4) * 2`

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	`(`	push (		(
2	3	output	3	(
3	+	push +	3	(+
4	4	output	3 4	(+
5	`)`	pop +; discard (	3 4 +	
6	*	push *	3 4 +	*
7	2	output	3 4 + 2	*
8	*end*	pop *	3 4 + 2 *	

The (is a wall: nothing pops it except its own). At row 5 the) releases the $+$. Compare with Trace 1: the same numbers, a different output, because the parentheses forced $+$ out **before** $\times$ arrived.

Question: What If We Never Pop on Equal?

Suppose the rule said "pop while the top has **higher** precedence" ($>$, not $\geq$). Run `8 - 3 - 2` again.

Q. In row 4 the new `-` just pushes (the old `-` stays). What postfix comes out, and what does it evaluate to?

#	Token	Action	Output (left to right)	Stack (bottom $\rightarrow$ top)
1	8	output	8	
2	-	push -	8	-
3	3	output	8 3	-
4	-	push only (no pop)	8 3	- -
5	2	output	8 3 2	- -
6	*end*	pop -, pop -	8 3 2 - -	

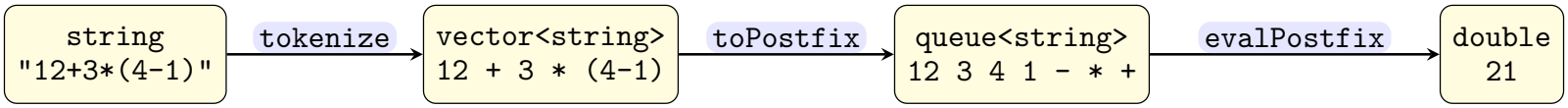
Answer. `8 3 2 - -` means $3 - 2 = 1$ first, then $8 - 1 = \mathbf{7}$, which is $8 - (3 - 2)$: **wrong** (the correct value is 3). Popping on equal precedence is what makes `-` and `/` left-associative.

Trace 3: Everything at Once, `5 + (1 + 2) * 4 - 3`

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	`5`	output	`5`	
2	`+`	push +	`5`	`+`
3	`(`	push (	`5`	`+ (`
4	`1`	output	`5 1`	`+ (`
5	`+`	push +	`5 1`	`+ ( +`
6	`2`	output	`5 1 2`	`+ ( +`
7	`)`	pop +; discard (	`5 1 2 +`	`+`
8	`*`	push *	`5 1 2 +`	`+ *`
9	`4`	output	`5 1 2 + 4`	`+ *`
10	`-`	pop *; pop +; push -	`5 1 2 + 4 * +`	`-`
11	`3`	output	`5 1 2 + 4 * + 3`	`-`
12	*end*	pop -	`5 1 2 + 4 * + 3 -`	

Row 10 is the busiest: the new - pops * (higher), then + (equal), then stops because the stack is empty. Result `5 1 2 + 4 * + 3 -` = 14 (evaluate it to check).

The Big Picture: Three Functions, One Pipeline



Think of an assembly line: each station takes the **output of the previous station** as its input.

- `tokenize` cuts the raw text into pieces (Tokenizer slides).
- `toPostfix` reorders the tokens so that operators come after their operands (today's main goal).
- `evalPostfix` reads that order with one stack and computes the value (earlier today).

In `main`: `evalPostfix(toPostfix(tokenize(text)))`. The types must match at every arrow, which is why `toPostfix` takes a `vector<string>` and returns a `queue<string>`.

Shunting-Yard, Part 1: The Function Header

```
std::queue<std::string> toPostfix(const std::vector<std::string>& tokens) {
    std::queue<std::string> output; // the postfix result, in order
    std::stack<std::string> ops;   // operators and "(" waiting here
    ...
}
```

Why these types? Read the arrows of the pipeline.

- **Parameter:** `tokenize` returned a `vector<string>`, so that is what we receive. `const &` means: look, don't copy, don't change.
- **Return:** `evalPostfix` wants a `queue<string>` (numbers and operators are both strings), so we return one.
- **`output` is a queue:** tokens must leave in the order they were added, like people in a line.
- **`ops` is a stack:** the most recent operator is the first one we compare with the next one (last in, first out).

Helpers (already written): `isOp(t)` is true for `+ - * /`; `prec(op)` is 2 for `* /` and 1 for `+ -`.

Shunting-Yard, Part 2: What to Do for an Operand

```
for (const std::string& t : tokens) {
    if (isOp(t)) {
        ...                         // Part 3
    } else if (t == "(") {
        ...                         // Part 3
    } else if (t == ")") {
        ...                         // Part 3
    } else {
        output.push(t);             // a number goes straight out
    }
}
```

Rule: a number never waits. Operands keep their original left-to-right order in postfix, so we send each one to `output` immediately.

Check: for `12 + 3` the output after reading `12` is `12`; after `+` it is still `12` (the `+` waits on `ops`); after `3` it is `12 3`.

Why is "number" the last `else`? Anything that is not an operator or a parenthesis must be a number, because `tokenize` produced only valid pieces (valid input assumed).

Shunting-Yard, Part 3a: What to Do for an Operator

```
if (isOp(t)) {
    while (!ops.empty() && isOp(ops.top())
           && prec(ops.top()) >= prec(t)) {
        output.push(ops.top()); // stronger or equal: leaves first
        ops.pop();
    }
    ops.push(t);                // then the new operator waits
}
```

In words: before the new operator `t` waits, every operator on top of the stack that must be done *first* goes to the output.

- `!ops.empty()`: nothing to compare with if the stack is empty.
- `isOp(ops.top())`: stop at a `(`; an opening parenthesis is a wall.
- `>=`: equal precedence also leaves, because `8 - 3 - 2` must be `(8 - 3) - 2` (left to right).

Question. With `>` instead of `>=`, `8 - 3 - 2` becomes `8 3 2 - -` (value 7, wrong).

Shunting-Yard, Part 3b: Parentheses

```
} else if (t == "(") {
    ops.push(t);                // remember where the group starts
} else if (t == ")") {
    while (ops.top() != "(") { // empty the group
        output.push(ops.top());
        ops.pop();
    }
    ops.pop();                  // discard the "(" itself
}
```

Analogy. (is a bookmark: it marks where a group begins. Reaching) means the group is finished, so we send everything above the bookmark to the output, then throw the bookmark away.

Important: neither parenthesis ever appears in the postfix output. Postfix order already says what is done first.

Check: (3 + 4) * 2. Read (: push. 3: output. +: top is (, so + just waits. 4: output.): pop + to output, discard (. *: stack is empty, so it waits. 2: output. End: pop *. Result: 3 4 + 2 *.

Shunting-Yard, Part 4: Flush the Stack

```
    }   // end of the for loop over tokens

    while (!ops.empty()) {      // the string is finished
        output.push(ops.top()); // everything still waiting
        ops.pop();              // goes out, top first
    }
    return output;
}
```

Why is this needed? Operators leave the stack only when a later token pushes them out. Nothing comes after the last token, so whatever is still waiting must be emptied by hand.

Example: `3 + 4 * 2`. After reading all tokens: output `3 4 2`, stack (top first) `* +`. Flush: pop `*`, then `+`. Result `3 4 2 * +`.

Why top first? The stack is LIFO. Operators still waiting must be written to the output from top to bottom to keep the required evaluation order.

Connecting the Pieces in `main`

```
int main() {
    std::string text = "12 + 3 * (4 - 1)";

    std::vector<std::string> tokens = tokenize(text);
        // {"12", "+", "3", "*", "(", "4", "-", "1", ")"}

    std::queue<std::string> postfix = toPostfix(tokens);
        // 12 3 4 1 - * +

    double answer = evalPostfix(postfix); // 21
    std::cout << answer << "\n";
}
```

Check yourself. Which of these would fail to compile?

- `evalPostfix(tokens)`: yes, a `vector` is not a queue.
- `toPostfix(text)`: yes, tokenize first.

Each function does one job, and its output type is exactly the next function's input type. Next: the complete code of `toPostfix`, then the typical mistakes.

Code: Shunting-Yard, All Four Parts Together

```
int prec(const std::string& op) {
    if (op == "*" || op == "/") { return 2; }
    return 1;                                          // "+" and "-"
}

std::queue<std::string> toPostfix(const std::vector<std::string>& tokens) {
    std::queue<std::string> output;
    std::stack<std::string> ops;
    for (const std::string& t : tokens) {
        if (isOp(t)) {
            while (!ops.empty() && isOp(ops.top()) && prec(ops.top()) >= prec(t)) {
                output.push(ops.top()); ops.pop();   // stronger or equal leaves
            }
            ops.push(t);
        } else if (t == "(") {
            ops.push(t);
        } else if (t == ")") {
            while (ops.top() != "(") { output.push(ops.top()); ops.pop(); }
            ops.pop();                                 // discard the "("
        } else {
            output.push(t);                            // a number goes straight out
        }
    }
    while (!ops.empty()) { output.push(ops.top()); ops.pop(); }
    return output;
}
```

Only the operator `while` loop encodes precedence; it stops at a `(`. **Valid-input version; validation omitted** (see Common Mistakes).

Common Mistakes

- **Operands popped in the wrong order** when evaluating: the **first** pop is the **right** operand.
- **Popping only on** $>$: $8 - 3 - 2$ becomes 7. Use $\geq$ for left-associative operators.
- **Forgetting the final flush**: after the last token, the stack still holds operators. They belong in the output.
- **Letting an operator pop a** `(`: the `(` must stay until its `)`. Check `isOp(ops.top())`.
- **No validation** (valid input assumed; Project 4 must add it). Not detected:
  - a `)` with no `(` (`top()` on an empty stack), or a leftover `(`;
  - fewer than two values for a binary operator, or not exactly one value at the end;
  - invalid tokens;
  - infix operand/operator order (infix `1 2 +` would slip through).
- **Tokenizing by single characters**: `12` must stay one token.

The complete, runnable code (with headers and `isOp`) is in `class18_expression_evaluator.cpp`.

Practice

1. Convert `2 * 3 + 4` to postfix.
2. Convert `9 - 4 - 3` to postfix and evaluate it.
3. Evaluate the postfix expression `5 1 2 + 4 * + 3 -`.
4. What is the largest number of values on the evaluation stack while evaluating `3 4 2 * +`?
5. Draw the expression tree of `2 * 3 + 4` and read it in post-order.
6. Tokenize, convert to postfix, and evaluate `12 / (2 + 1) - 1`.

Optional: Evaluate a Postfix String Directly

Optional alternative implementation (not required for Exercise 8). Exercise 8 requires the queue-based `evalPostfix`; this section shows the same stack principle with a different input interface (a string). If the input is **already postfix**, `postfixProcess` scans the string and evaluates it with **one stack**, skipping the tokenizer and queue. It does *not* replace shunting-yard conversion of infix input.

```
// Evaluate postfix text using one operand stack.
// Nonnegative integer literals and binary + - * /.
double postfixProcess(const std::string& s);

int main() {
  std::cout << postfixProcess("7 2 /") << '\n';   // 3.5
  std::cout << postfixProcess("20 6 2 - /") << '\n'; // 5
}
```

Q. Compared with tokenize + queue, what is the trade-off?

(A) The stack work is different: the answer may differ.

(B) **The stack work is the same, but reading characters and evaluating are now mixed in one function.**

(C) It works for infix input too.

(D) It needs a queue as well.

Answer: (B). Same rule (number: push; operator: pop two, compute, push). The function is shorter, but it must now rebuild multi-digit numbers itself: the next slides.

Optional: The Outer Structure

The next slides fill in the three branches. First, the frame that holds them:

```
double postfixProcess(const std::string& s) {
    std::stack<double> operands;  // the one stack
    std::size_t i = 0;            // our own index
    while (i < s.length()) {
        // digit branch:  build a number, push it, then continue
        // operator branch: pop two values, compute, push the result
        // other branch:  skip white space, reject any other character
        i++;                      // skipped by the digit branch
    }
    // after the loop: exactly one value must remain; return it
}
```

Notice. The function declares the stack and the index **before** the loop, and checks the stack **after** it. The digit branch moves `i` itself (inside its inner loop), which is why it must not reach the `i++`.

Optional: Building a Number Digit by Digit

There is no `tokenize`, so the function must turn the characters `'3'`, `'4'`, `'5'` into the number 345.

```
if (std::isdigit(static_cast<unsigned char>(s[i]))) {
  double num = 0;
  while (i < s.length() && std::isdigit(static_cast<unsigned char>(s[i]))) {
    num = num * 10 + (s[i] - '0');  // append one digit
    i++;
  }
  operands.push(num);
  continue;     // i already points past the number
}
```

`s[i]`	`s[i] - '0'`	`num` before	`num * 10 + (s[i] - '0')`
`'3'`	3	0	$0 \cdot 10 + 3 = 3$
`'4'`	4	3	$3 \cdot 10 + 4 = 34$
`'5'`	5	34	$34 \cdot 10 + 5 = 345$

Why `s[i] - '0'`? A `char` is a small integer code, and the digit characters `'0'`...`'9'` are consecutive, so `'7' - '0'` equals 7, the *value* of the digit. **Why `num * 10`?** Each new digit shifts the earlier digits one place to the left.

Optional: Why `continue` After a Number?

At the bottom of the outer `while` loop, every other branch (a space, an operator) ends with `i++`. The digit branch is different: the inner `while` has **already moved** `i` to the first character after the number. Trace `"2 3+"`:

- The digit 3 is at `i = 2`. The inner `while` reads it, adds 1 to `i`, and stops at `i = 3`: the `'+'`.
- `continue` jumps back to the loop test with `i = 3`, so the `'+'` is processed next.

Q. Remove the line `continue;`. What happens to `"2 3+"`?

(A) Nothing changes.
(B) **The `'+'` is skipped: the `i++` at the bottom moves `i` from 3 to 4, so the stack ends with two values (error: extra operands).**
(C) The program prints 5 twice.
(D) The digit 3 is read twice.

Answer: (B). Without `continue`, the digit branch falls through to the `i++` at the bottom and steps over the character after the number. In `"12 3 + 2 *"` the bug would stay hidden, because only spaces follow the numbers: a symbol **right after** a number exposes it. (An alternative is to write `i--` after the inner loop; `continue` avoids the back-step.)

Optional: An Operator

```
else if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {
  if (operands.size() < 2) {
    throw std::runtime_error(std::string("need two operands for operator ") + s[i]);
  }
  double num2 = operands.top(); // first pop: the RIGHT operand
  operands.pop();
  double num = operands.top(); // second pop: the LEFT operand
  operands.pop();
  switch (s[i]) {
    case '+': operands.push(num + num2); break;
    case '-': operands.push(num - num2); break;
    case '*': operands.push(num * num2); break;
    case '/':
      if (num2 == 0) { throw std::runtime_error("division by zero"); }
      operands.push(num / num2);
      break;
  }
}
```

Q. Why does the code test `operands.size() < 2` *before* popping?

(A) To make the code faster.

(B) **Calling `top()` or `pop()` on an empty stack is undefined behavior, so an input like `"7 +"` must be rejected first.**

(C) Because `switch` needs two values.

(D) Because `double` cannot be popped twice.

Answer: (B). Notice that `num2` (the *first* pop) is the right operand, so the subtraction is `num - num2`.

Optional: Bad Input and the Final Check

```
    else if (!std::isspace(static_cast<unsigned char>(s[i]))) {
      throw std::runtime_error(std::string("invalid character: ") + s[i]);
    }
    i++;                                // every branch except the digit branch
  }                                     // end of the while loop

  if (operands.empty()) { throw std::runtime_error("missing operand"); }
  double value = operands.top();
  operands.pop();
  if (!operands.empty()) {
    throw std::runtime_error(std::string("extra operands: ") + s);
  }
  return value;
}
```

Q. What does `postfixProcess` do for each input?

Input	Your prediction	Result
`"7 2 &"`		error: invalid character &
`"7 +"`		error: need two operands for operator +
`""`		error: missing operand
`"7 2"`		error: extra operands: 7 2
`"12 3 + 2 *"`		30

A valid postfix expression leaves **exactly one** value on the stack. Code file: `evaluate_postfix_string.cpp`.

Summary and Preview

- **Postfix** needs no parentheses or precedence: one stack evaluates it.
- **Shunting-yard**: numbers go straight out; operators **wait on a stack**, and leave when precedence and associativity require it, at a closing parenthesis, or in the final flush. For n tokens, each operator is pushed and popped once: $O(n)$ time and $O(n)$ auxiliary space.
- **Parentheses** are walls on the stack; precedence is the pop rule.
- An **expression tree** shows the structure; its traversals are the three notations.

Coming up

Nov 10: searching: linear search, binary search, and why sorted data matters.
Nov 12: Exercise 8 (Shunting-Yard Trace) due.
Later: Project 4 (Expression Evaluator) puts tokenizing, conversion, and evaluation together.